

머신러닝

넘파이

Chapter 3



Prof. Yang



한국성서대학교
KOREAN BIBLE UNIVERSITY

목 차

CONTENTS

01 넘파이란?

02 넘파이 배열 객체 다루기

03 넘파이 배열 연산

04 비교 연산과 데이터 추출

01 넴파이란?



넘파이의 개념

- 파이썬의 고성능 과학 계산을 라이브러리
- 벡터나 행렬 같은 선형대수의 표현법을 코드로 처리
- 사실상의 표준 라이브러리
- 다차원 리스트나 크기가 큰 데이터 처리에 유리

$$\begin{array}{c} \text{Feature Data} \\ \left[\begin{array}{ccc} 1 & 2 & 3 \\ 4 & 5 & 6 \end{array} \right] \end{array} \begin{array}{c} W \\ \left[\begin{array}{c} 1 \\ 1 \\ 0 \end{array} \right] \end{array} = \left[\begin{array}{c} 3 \\ \end{array} \right]$$

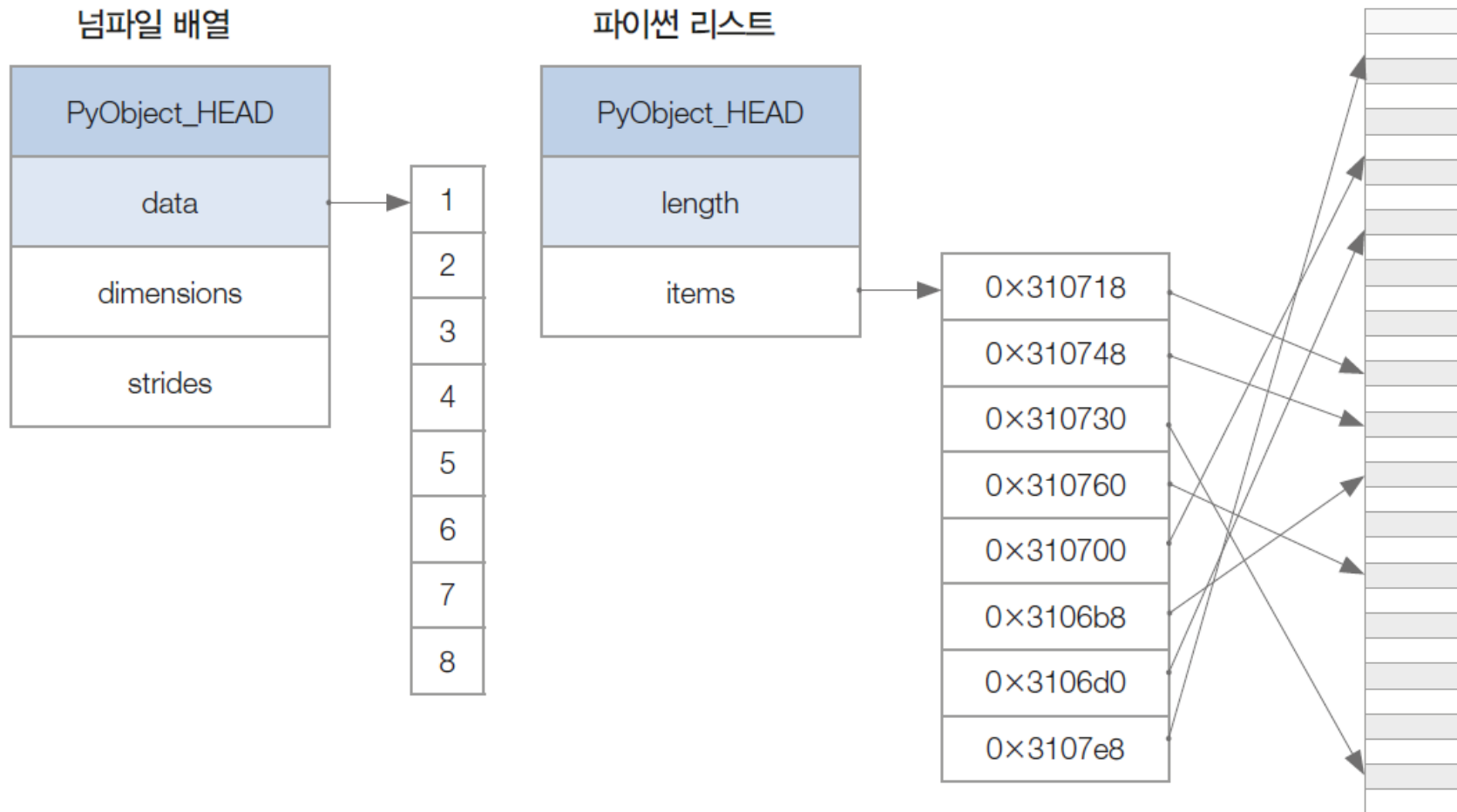
$$\begin{array}{c} \left[\begin{array}{ccc} 1 & 2 & 3 \\ 4 & 5 & 6 \end{array} \right] \end{array} \begin{array}{c} \left[\begin{array}{c} 1 \\ 1 \\ 0 \end{array} \right] \end{array} = \left[\begin{array}{c} 3 \\ 9 \end{array} \right]$$



넘파이의 특징

- 속도가 빠르고 메모리 사용이 효율적
 - 데이터를 메모리에 할당하는 방식이 기존과 다름
- 반복문을 사용하지 않음
 - 연산할 때 병렬로 처리
 - 함수를 한 번에 많은 요소에 적용
- 다양한 선형대수 관련 함수 제공
- C, C++, 포트란 등 다른 언어와 통합 사용 가능

▶ 넘파이 배열 vs 파이썬 기본 배열(리스트)



파이썬에서 기본적으로 제공하는 배열은 사실 리스트 형식으로 값을 저장함

▶ 넘파이 배열 vs 파이썬 기본 배열(리스트)

- 파이썬 리스트와 넘파이 배열의 차이점

- 텐서 구조에 따라 배열 생성

- 배열의 모든 구성 요소에 값이 존재해야 함

```
import numpy as np
test_list = [[1, 4, 5, 8], [1, 4, 5]]
np.array(test_list, float) # ValueError
```

- 동적 데이터 타입을 지원하지 않음

- 하나의 데이터 타입만 사용

- 데이터를 메모리에 연속적으로 나열

- 각 값 메모리 크기가 동일
- 검색이나 연산 속도가 리스트에 비해 훨씬 빠름

02

넘파이 배열 객체 다루기



넘파이 배열과 텐서

- 넘파이 배열(ndarray) : 넘파이에서 텐서 데이터를 다루는 객체

- 텐서(tensor) : 선형대수의 데이터 배열

- 랭크(rank)에 따라 이름이 다름
- 랭크=차원

랭크(Rank)	이름	예
0	스칼라(scalar)	7
1	벡터(vector)	[10, 10]
2	행렬(matrix)	[[10, 10], [15, 15]]
3	3차원 텐서(3-order tensor)	[[[1, 5, 9], [2, 6, 10]], [[3, 7, 11], [4, 8, 12]]]
n	n차원 텐서(n-order tensor)	

- np.array 함수 사용하여 배열 생성

- 매개변수 1: 배열 정보 (정확히는 list타입)
- 매개변수 2: 넘파이 배열로 표현하려는 데이터 타입

```
import numpy as np
test_array = np.array([1, 4, 5, 8], float)
print(test_array)
```

[1. 4. 5. 8.]



배열 생성 코드

- 실제 메모리에 저장된 데이터의 형태와 의미

```
import numpy as np #numpy 라이브러리 호출  
tmp = [[1, 4, 5, 8], [21, 23, 15, 11]] #리스트 형식의 2차 배열  
test_array = np.array(tmp, float) #2차원 배열 및 형 변환  
test_array # 'test_array' 출력
```

```
array([[ 1.,  4.,  5.,  8.],  
       [21., 23., 15., 11.]])
```

Element
number

0	1	2	3	0	1	2	3
1	4	5	8	21	23	15	11

Group
number

```
print(type(tmp))  
print(type(tmp[1][3]))
```

```
<class 'list'>  
<class 'int'>
```

```
print(type(test_array[0]))  
print(type(test_array[1][3]))
```

```
<class 'numpy.ndarray'>  
<class 'numpy.float64'>
```



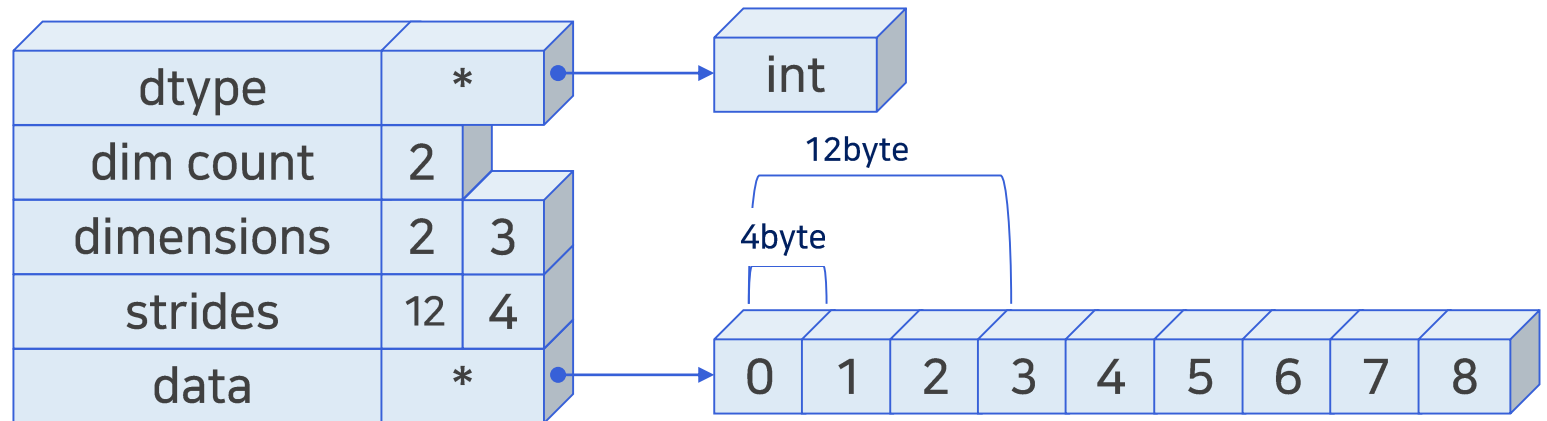
넘파이 배열 데이터 구조

```
test_array = np.array([[0,1,2], [3,4,5], [6,7,8]])  
print(test_array)
```

```
[[0 1 2]  
 [3 4 5]  
 [6 7 8]]
```

```
print(test_array.dtype)    # 데이터의 자료형  
print(test_array.ndim)    # 랭크=차원  
print(test_array.shape)   # 구조  
print(test_array.strides) # 메모리 간격(Byte 단위)
```

```
int32  
2  
(3, 3)  
(12, 4)
```





벡터와 매트릭스

```
vector = np.array([1, 4, 5, "8"], float)
print(vector)
```

```
[1.  4.  5.  8.]
```

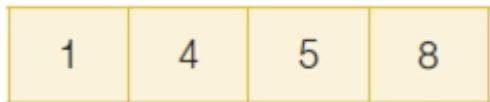
```
print(vector.dtype)  #자료형
print(vector.ndim)   #차원=랭크
print(vector.shape)  #구조
print(vector.strides) #크기
```

```
float64
```

```
1
```

```
(4,)
```

```
(8,)
```



```
(4,)
```

넘파이 배열(ndarray)의 구성

넘파이 배열의 구조(shape)
(타입은 튜플)

```
matrix = np.array([[1,2,5,8], [1,2,5,8], [1,2,5,8]])
print(matrix)
```

```
[[1 2 5 8]
```

```
 [1 2 5 8]
```

```
 [1 2 5 8]]
```

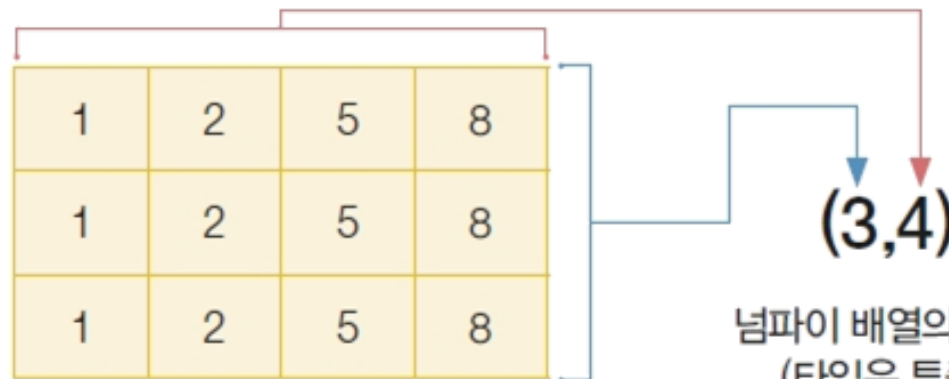
```
print(matrix.dtype)  #자료형
print(matrix.ndim)   #차원=랭크
print(matrix.shape)  #구조
print(matrix.strides) #크기
```

```
int32
```

```
2
```

```
(3, 4)
```

```
(16, 4)
```



```
(3,4)
```

넘파이 배열의 구조
(타입은 튜플)



구조 변경 및 랭크 조절: reshape

- reshape 함수로 배열의 구조를 변경하고 랭크를 조절



```
x = np.array([[1, 2, 5, 8], [1, 2, 5, 8]])  
x.shape
```

```
(2, 4)
```

```
x.reshape(-1,)
```

```
array([1, 2, 5, 8, 1, 2, 5, 8])
```

```
print(x)
```

```
[[1 2 5 8]  
 [1 2 5 8]]
```

-1을 사용하면 나머지 차원의 크기를 지정했을 때 전체 요소의 개수를 고려하여 마지막 차원이 자동으로 지정됨



구조 변경 및 랭크 조절: reshape

- 반드시 전체 요소의 개수는 동일

```
x = np.array(range(8))  
x
```

```
array([0, 1, 2, 3, 4, 5, 6, 7])
```

```
x.reshape(2,2)
```

ValueError

Traceback (most recent call last)

Cell In[23], line 1

```
----> 1 x.reshape(2,2)
```

ValueError: cannot reshape array of size 8 into shape (2,2)

```
x.reshape(4,2) #but x의 모양은 그대로임
```

```
array([[0, 1],  
       [2, 3],  
       [4, 5],  
       [6, 7]])
```

```
x.reshape(2,4) #but x의 모양은 그대로임
```

```
array([[0, 1, 2, 3],  
       [4, 5, 6, 7]])
```

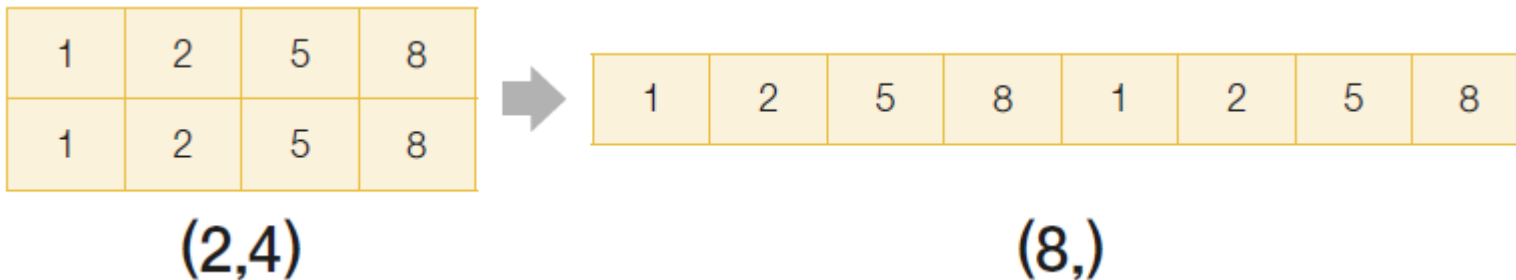
range(n)함수

- 특정 구간의 숫자의 범위를 만들어주는 함수
- 0~n-1까지 n개의 원소로 구성된 숫자열을 **range타입**으로 반환함



구조 변경 및 랭크 조절: flatten

- flatten 함수는 데이터 그대로 1차원으로 변경
 - 데이터의 개수는 그대로 존재
 - 배열의 구조만 변함



```
x = np.array([[1, 2, 5, 8], [1, 2, 5, 8]])  
x
```

```
array([[1, 2, 5, 8],  
       [1, 2, 5, 8]])
```

```
x.flatten() #but x의 모양은 그대로임
```

```
array([1, 2, 5, 8, 1, 2, 5, 8])
```



인덱싱

- 넘파이 배열의 인덱스 표현에는 ','을 지원

- '[행][열]' 또는 '[행,열]' 형태

- 구조: 행,열 자리에는 스칼라값 혹은 벡터 값을 사용할 수 있음

- 자료형: int형 or boolean형 가능

```
x = np.array(range(6)).reshape(2,3)  
x
```

```
array([[0, 1, 2],  
       [3, 4, 5]])
```

```
x[0][0]
```

```
0
```

```
x[0,2]
```

```
2
```

```
x[0, 1] = 100
```

```
x
```

```
array([[ 0, 100,  2],  
       [ 3,  4,  5]])
```




인덱싱

- ":" 사용 => 벡터를 의미
 - 시작인덱스:끝인덱스-1
 - ":"은 전체를 의미함

- 예)아래 구조의 x매트릭스 사용

0	1	2	3	4
5	6	7	8	9

```
x = np.array(range(10)).reshape(2,5)
```

```
x[:,2:] # 전체 행의 2열 이상
```

```
array([[2, 3, 4],  
       [7, 8, 9]])
```

```
x[1,1:3] # 1행의 1열 ~ 2열
```

```
array([6, 7])
```

```
x[1:3] # 1~2행 전체=> 2행은 없으므로 1행만
```

```
array([[5, 6, 7, 8, 9]])
```

```
x[:,[1,3,4]]
```

```
array([[1, 3, 4],  
       [6, 8, 9]])
```



arange

- range 함수와 같이 차례대로 값을 생성하여 넘파이 배열 객체를 반환
- '(시작 인덱스, 마지막 인덱스, 증가값)'으로 구성
- range 함수와 달리 증가값에 실수형이 입력되어도 값을 생성할 수 있음
- 소수점 값을 주기적으로 생성할 때 유용

```
np.arange(10)
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
np.arange(-5, 5)
```

```
array([-5, -4, -3, -2, -1, 0, 1, 2, 3, 4])
```

```
np.arange(0, 5, 0.5)
```

```
array([0. , 0.5, 1. , 1.5, 2. , 2.5, 3. , 3.5, 4. , 4.5])
```



ones, zeros, empty

- ones 함수 : 1로만 구성된 넘파이 배열을 생성
- zeros 함수 : 0으로만 구성된 넘파이 배열을 생성
- empty 함수 지정한 shape 크기의 메모리를 할당하여 배열을 생성 (초기화하지 않음)
 - ones와 zeros는 먼저 shape의 크기만큼 메모리를 할당하고 그곳에 값을 채움
 - empty는 메모리 초기화하지 않아 생성될 때마다 예측되지 않은 값 반환
- 생성 시점에서 dtype을 지정해주면 해당 데이터 타입으로 배열 생성



ones, zeros, empty

```
np.ones(shape=(5,2), dtype=np.int8)
```

```
array([[1, 1],  
       [1, 1],  
       [1, 1],  
       [1, 1],  
       [1, 1]], dtype=int8)
```

```
np.zeros(shape=(2,2), dtype=np.float32)
```

```
array([[0., 0.],  
       [0., 0.]], dtype=float32)
```

```
np.empty(shape=(2,4), dtype=np.float32)
```

```
array([[0.000000e+00, 1.401298e-45, 0.000000e+00, 7.806146e-39],  
       [1.788057e-42, 0.000000e+00, 1.076197e-42, 9.388700e-44]],  
      dtype=float32)
```



랜덤함수

- uniform 함수 : 균등분포 함수

- 'np.random.uniform(시작값, 끝값, 데이터개수)'

```
np.random.uniform(0, 5, 10)
```

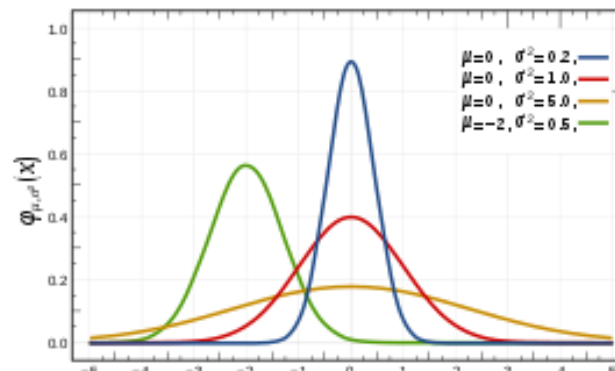
```
array([2.10688707, 1.19574991, 0.35939211, 0.39610817, 2.74490792,  
       0.27413755, 1.59647365, 4.73022193, 0.23329135, 0.97507957])
```

- normal 함수 : 정규분포 함수

- 'np.random.normal(평균값, 표준편차, 데이터개수)'

```
np.random.normal(0, 2, 10)
```

```
array([ 0.53887261,  1.07569963,  0.23875709, -2.95222671, -0.80287842,  
       -1.49589927,  1.95811855, -4.09813681,  1.26469919,  1.199728  ])
```



- Ch02 데이터의 이해 p.29 np.random.choice참고

03

넘파이 배열 연산



연산 함수: sum

- 연산 함수(operation function) : 배열 내부 연산을 지원하는 함수

- np.함수명(배열이름, 축 방향)
- 배열이름.함수명(축 방향)

랭크가 2 이상인 배열에 적용할 때 축으로 연산의 방향을 설정

- sum 함수 : 요소의 합을 반환

```
import numpy as np
test_array = np.arange(1, 13) #벡터
test_array.sum()
```

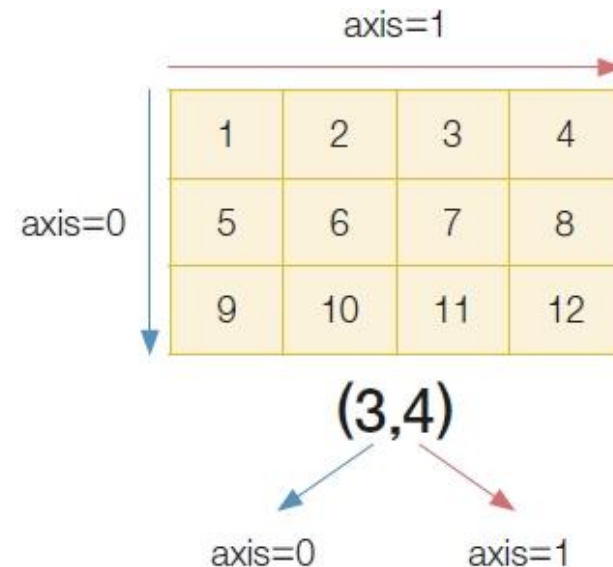
78

```
test_array = test_array.reshape(3,4)
test_array.sum(axis=0) #np.sum(test_array, axis=0)
```

array([15, 18, 21, 24])

```
test_array.sum(axis=1) #np.sum(test_array, axis=1)
```

array([10, 26, 42])





연산 함수: mean, std, sqrt

- mean 함수 : 요소 평균 반환
- std 함수 : 요소의 표준편차를 반환
- sqrt 함수 : 각 요소의 제곱근을 반환

```
test_array.mean(axis=1) # axis=1 축을 기준으로 평균 연산
```

```
array([ 2.5,  6.5, 10.5])
```

```
np.std(test_array) # 전체 값에 대한 표준편차 연산
```

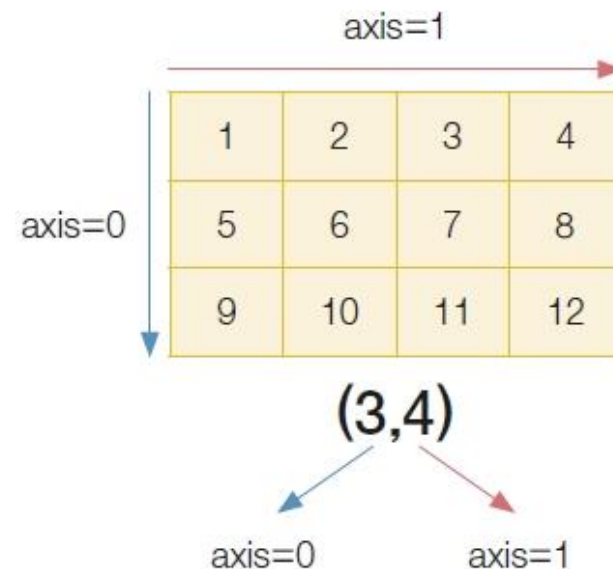
```
3.452052529534663
```

```
test_array.std(axis=0) # axis=0 축을 기준으로 표준편차 연산
```

```
array([3.26598632, 3.26598632, 3.26598632, 3.26598632])
```

```
np.sqrt(test_array) # 각 요소에 제곱근 연산 수행
```

```
array([[1.          , 1.41421356, 1.73205081, 2.          ],  
       [2.23606798, 2.44948974, 2.64575131, 2.82842712],  
       [3.          , 3.16227766, 3.31662479, 3.46410162]])
```





연산 함수: var

- var 함수 : 각 요소의 분산을 반환

```
import numpy as np #numpy라이브러리호출
```

```
tmp= [[1, 4, 5, 8], [21, 23, 15, 11]] #리스트형식의2차배열  
test_array = np.array(tmp, float)#2차원배열및형변환
```

```
print(test_array) #'test_array' 출력
```

```
print(test_array.var())
```

```
print(test_array.var(0))#axis=0
```

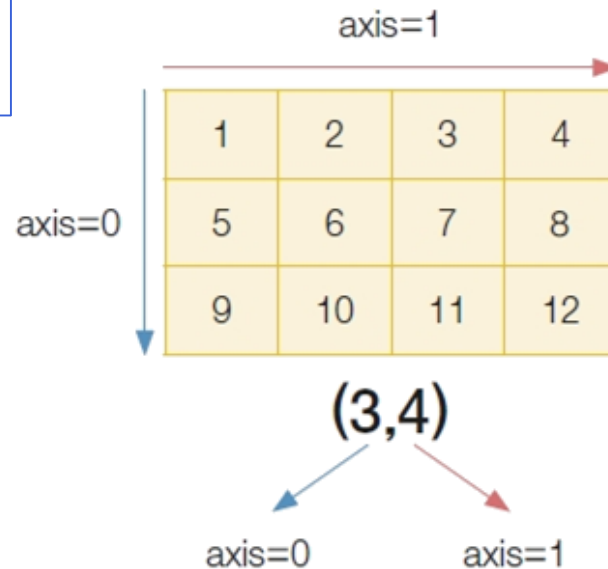
```
print(test_array.var(1))#axis=1
```

```
[[ 1.  4.  5.  8.]  
 [21. 23. 15. 11.]]
```

```
56.75
```

```
[100.    90.25  25.      2.25]
```

```
[ 6.25 22.75]
```





연결 함수

■ 연결 함수(Concatenation Functions) : 두 객체 간의 결합을 지원하는 함수

- vstack 함수 : 배열을 수직으로 붙여 하나의 행렬을 생성
- hstack 함수 : 배열을 수평으로 붙여 하나의 행렬을 생성

```
v1 = np.array([1, 2, 3])  
v2 = np.array([4, 5, 6])  
np.vstack((v1, v2))
```

```
array([[1, 2, 3],  
       [4, 5, 6]])
```

```
np.hstack((v1, v2))
```

```
array([1, 2, 3, 4, 5, 6])
```

```
v1 = v1.reshape(-1, 1)  
v2 = v2.reshape(-1, 1)  
np.hstack((v1, v2))
```

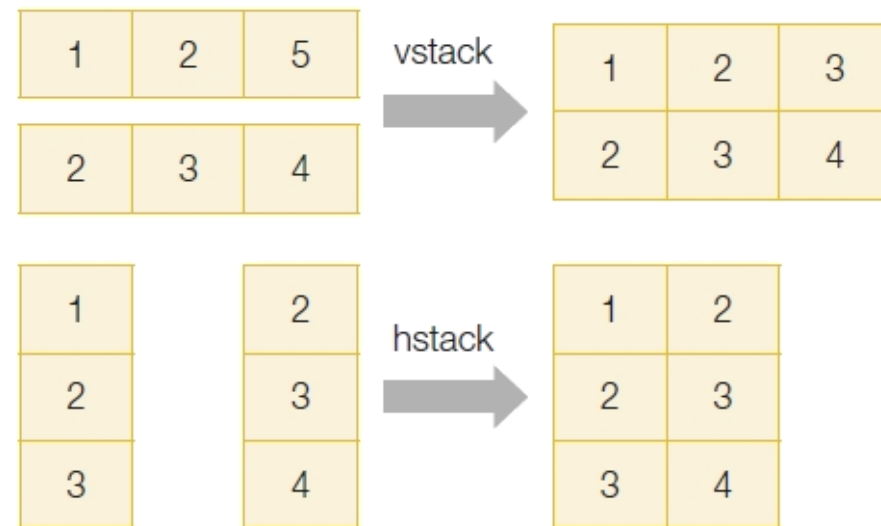


그림 3-14 연결을 통한 행렬의 결합



```
array([[1, 4],  
       [2, 5],  
       [3, 6]])
```



사칙연산

- 넘파이는 파이썬과 동일하게 배열 간 사칙연산 지원
 - 행렬과 행렬, 벡터와 벡터 간 사칙연산이 가능
- 같은 배열의 구조일 때 요소별 연산(element-wise operation)
 - 요소별 연산 : 기본적으로 두 배열의 구조가 동일할 경우 같은 인덱스 요소들끼리 연산

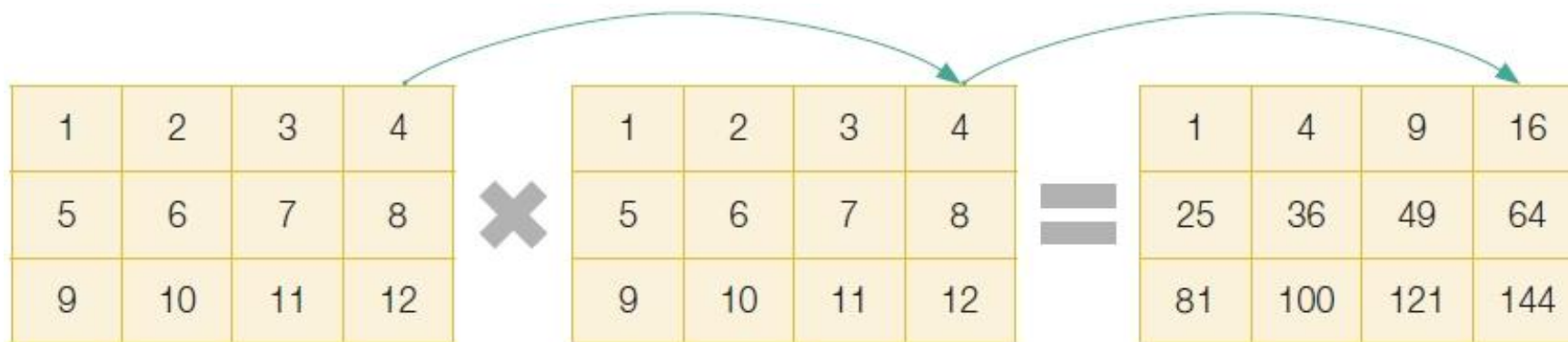


그림 3-16 배열 간 요소별 연산의 예시



사칙연산

```
x = np.arange(1, 7).reshape(2,3)
```

x

```
array([[1, 2, 3],  
       [4, 5, 6]])
```

x+x

```
array([[ 2,  4,  6],  
       [ 8, 10, 12]])
```

x-x

```
array([[0, 0, 0],  
       [0, 0, 0]])
```

x/x

```
array([[1., 1., 1.],  
       [1., 1., 1.]])
```

x**x

```
array([[ 1,  4, 27],  
       [256, 3125, 46656]])
```



벡터 내적(곱)

- 배열 간의 곱셈에서는 요소별 연산과 벡터의 내적(dot product) 연산 가능
 - 벡터의 내적 : 두 배열 간의 곱셈

```
x_1 = np.arange(1, 7).reshape(2,3)
x_2 = np.arange(1, 7).reshape(3,2)
print(x_1)
print(x_2)
```

```
[[1 2 3]
 [4 5 6]]
[[1 2]
 [3 4]
 [5 6]]
```

```
x_1.dot(x_2)
```

```
array([[22, 28],
       [49, 64]])
```



브로드캐스팅 연산(broadcasting operations)

- 브로드캐스팅 연산(broadcasting operations) :

하나의 행렬과 스칼라 값들 간의 연산이나 행렬과 벡터 간의 연산

- 방송국의 전파가 퍼지듯 뒤에 있는 스칼라 값이 모든 요소에 퍼지듯이 연산

```
x = np.arange(1, 10).reshape(3,3)
x
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

```
x + 10
```

```
array([[11, 12, 13],
       [14, 15, 16],
       [17, 18, 19]])
```

```
x + 2
```

```
array([[ -1,  0,  1],
       [  2,  3,  4],
       [  5,  6,  7]])
```



브로드캐스팅 연산(broadcasting operations)

- 행렬과 스칼라 값 외에 매트릭스와 벡터 간에도 연산

1	2	3
4	5	6
7	8	9
10	11	12

x

+

10	20	30
----	----	----

v

=

11	22	33
14	25	36
17	28	39
20	31	42

```
x = np.arange(1, 13).reshape(4,3)
v = np.arange(10, 40, 10)
x + v
```

```
array([[11, 22, 33],
       [14, 25, 36],
       [17, 28, 39],
       [20, 31, 42]])
```

04

비교 연산과 데이터 추출



브로드캐스팅 비교 연산

- 비교연산 : 연산 결과는 항상 불린형(boolean type)을 가진 배열로 추출
- 브로드캐스팅 비교 연산 : 하나의 스칼라 값과 벡터 간의 비교 연산은 벡터 내 전체 요소에 적용

```
import numpy as np
x = np.array([4, 3, 2, 6, 8, 5])
x > 3
```

```
array([ True, False, False,  True,  True,  True])
```

- 두 개의 배열 간 배열의 구조(shape)가 동일한 경우
 - 같은 위치에 있는 요소들끼리 비교 연산
 - $[1 > 2, 3 > 1, 0 > 7]$ 과 같이 연산이 실행한 후 이를 반환

```
x = np.array([1, 3, 0])
y = np.array([2, 1, 7])
x > y
```

```
array([False,  True, False])
```



불린 인덱스

- 불린 인덱스(boolean index) : 배열에 있는 값들을 반환할 특정 조건을 불린형의 배열에 넣어서 추출
 - 인덱스에 들어가는 배열은 불린형이어야 함
 - 불린형 배열과 추출 대상이 되는 배열의 구조가 같아야 함

```
x = np.array([4, 6, 7, 3, 2])  
x > 3
```

```
array([ True,  True,  True, False, False])
```

```
cond = x > 3  
x[cond] # x[x>3]
```

```
array([4, 6, 7])
```



all과 any

- all 함수 : 배열 내부의 모든 값이 참일 때는 True, 하나라도 참이 아닐 경우에는 False를 반환
 - and 조건을 전체 요소에 적용
- any 함수 : 배열 내부의 값 중 하나라도 참일 때는 True, 모두 거짓일 경우 False를 반환 or 조건을 전체 요소에 적용

```
x = np.array([4, 3, 2, 6, 8, 5])  
(x > 3).all()
```

False

```
(x > 3).any()
```

True



where 함수

- where 함수 : 배열이 불린형으로 이루어졌을 때 참인 값들의 인덱스를 반환

```
x = np.array([4, 6, 7, 3, 2])  
x > 5
```

```
array([False,  True,  True, False, False])
```

```
np.where(x>5)
```

```
(array([1, 2], dtype=int64),)
```

- $x > 5$ 를 만족하는 값은 6과 7
- 6과 7의 인덱스 값인 [1, 2]를 반환

- True/False 대신 참/거짓인 경우의 값을 지정할 수 있음

```
x = np.array([4, 6, 7, 3, 2])  
np.where(x>5 , 10, 20)
```

```
array([20, 10, 10, 20, 20])
```

- 참일 경우에 10, 거짓일 경우에 20을 반환



해당 인덱스 반환 함수

- `argsort` : 배열 내 값들을 작은 순서대로 인덱스를 반환
- `argmax` : 배열 내 값들 중 가장 큰 값의 인덱스를 반환
- `argmin` : 배열 내 값들 중 가장 작은 값의 인덱스를 반환

```
x = np.array([4, 6, 7, 3, 2])  
np.argsort(x)
```

```
array([4, 3, 0, 1, 2], dtype=int64)
```

```
np.argmax(x)
```

2

```
np.argmin(x)
```

4



unique함수

- 배열에서 중복된 값을 제거하고 고유한 값만 반환
- 결과는 자동으로 정렬됨

```
gender = np.array(['M', 'F', 'F', 'M', 'M', 'F', 'F', 'M'])
```

```
values = np.unique(gender)  
values
```

```
array(['F', 'M'], dtype='<U1')
```

```
values, counts = np.unique(gender, return_counts=True) #값 개수까지 반환
```

```
print(values)  
print(counts)
```

```
['F' 'M']  
[4 4]
```



SUMMARY

- 넘파이 배열의 개념과 특징
 - .dtype : 자료형
 - .ndim : 차원(랭크)
 - .shape : 구조
 - .strides : 크기
 - .size : 전체크기
 - .reshape함수:구조 및 차원 조절
 - .flatten함수:1차원으로 변경
- 넘파이 제공 함수: arange, one, zeros, empty, random함수
- 넘파이를 이용하여 넘파이 배열 연산을 수행
 - 연산함수 : sum, mean, std, sqrt
 - 연결함수 : vstack, hstack
 - 내적곱 : dot
- 넘파이를 이용하여 비교 연산과 데이터 추출 방법을 이해
 - 비교연산 : all, any
 - 조건함수: where
 - 해당 값의 인덱스:argmax, argmin, argsort
 - 값 종류 추출: unique



Q&A

궁금한 사항은 아래 방법으로 문의 바랍니다.



이메일
dana1112@bible.ac.kr



LMS 쪽지

THANK YOU FOR YOUR ATTENTION!

한국성서대학교 | 인공지능융합학부 | 머신러닝